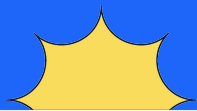




Agenda



¿Qué son los agentes?

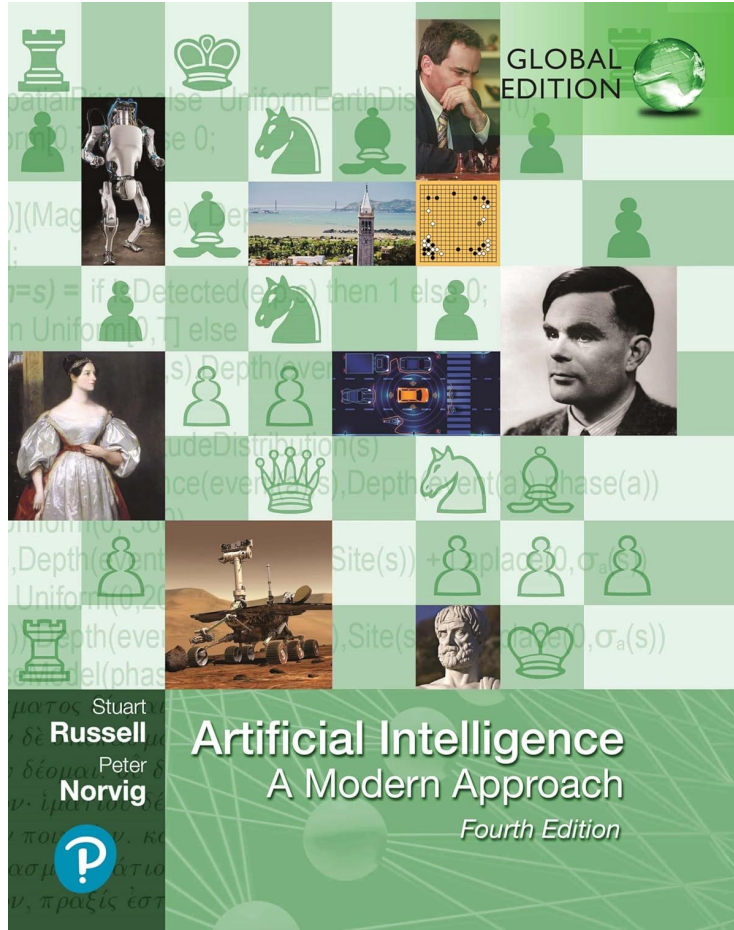
Cómo funcionan los agentes

Algunos ejemplos prácticos

MCPs. Qué son y ejemplos prácticos

¿Por dónde empezar?

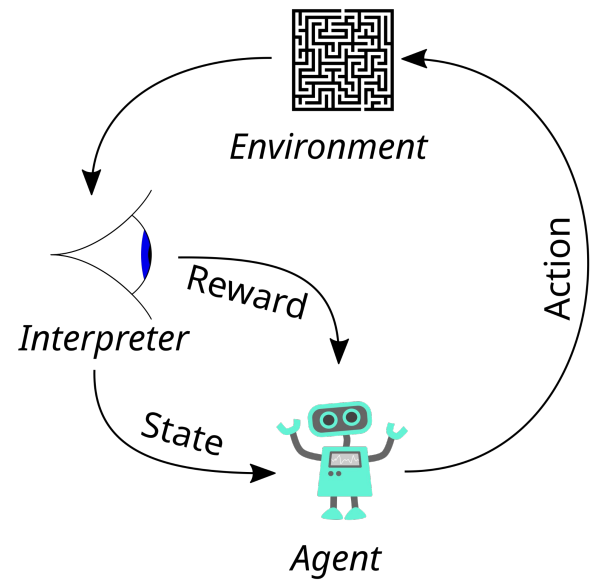


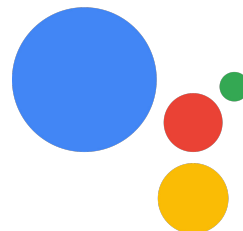


IA

“the designing and building of intelligent agents that receive percepts from the environment and take actions that affect that environment”

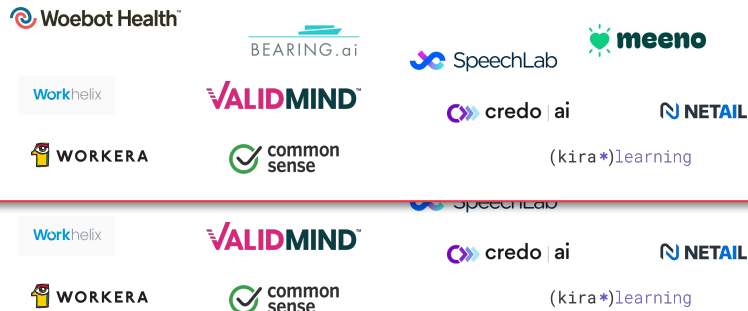






**Un agente es una aplicación
cuyo flujo de control está
controlado por un LLM**

Aplicaciones



Foundation Models

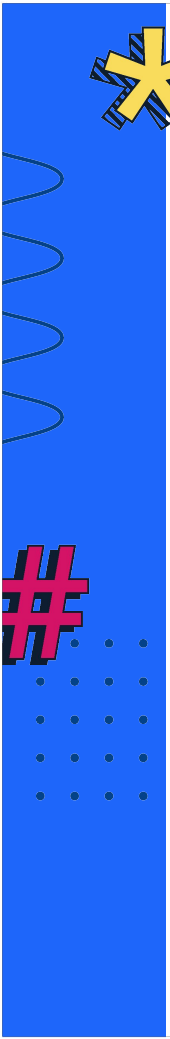


Infraestructura Cloud



Hardware



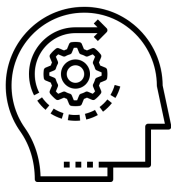


Como funcionan los agentes

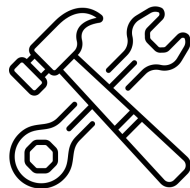
**Un agente es una aplicación
cuyo flujo de control está
controlado por un LLM**

		Decide output of step	Decide which step to take	Decide what steps are available to take
AGENTS	CODE			
	LLM CALL			
	CHAIN			
	ROUTER			
	ReACT			
	AUTONOMOUS			

CAPACIDADES



REASONING



TOOL CALLING



MEMORY

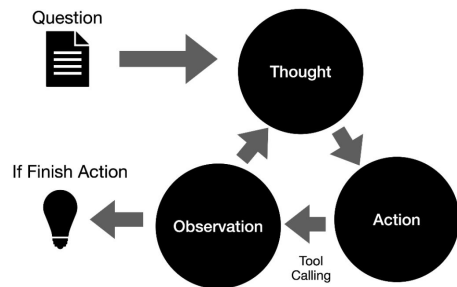


ReAct PARADIGM

Combina Razonamiento y Acción, lo que permite a los agentes resolver tareas de forma inteligente alternando entre pensar (razonar) y actuar.

Cómo funciona:

- **Razonamiento:** El LLM procesa la entrada y genera un pensamiento o plan basado en su análisis. Este paso permite al agente determinar la siguiente acción o herramienta que debe emplear. Un LLM puede decidir cuál de muchas herramientas debe invocar.
- **Acción:** Basándose en la fase de razonamiento, el agente llama a una herramienta o ejecuta una tarea. Esto puede implicar consultar una API, recuperar datos o ejecutar una función específica.
- **Iteración:** El agente continúa alternando entre razonar y actuar, afinando su enfoque en cada ciclo hasta que la tarea se completa o ya no se requieren más acciones.



Answer the following questions as best as you can. You have access to the following tools:

{tools}

Use the following format:

Question: the input question you must answer

Thought: you should always think about what to do

Action: the action to take, should be one of {tools}

Action Input: the input to the action

Observation: the result of the action

... (this Thought/Action/Action Input/Observation can repeat N times)

Thought: I now know the final answer

Final Answer: the final answer to the original input question

Begin!

Question: {input}

Thought: {agent_scratchpad}



Tools

Son herramientas que permiten ampliar la capacidad de los LLMs. Inicialmente consistían en llamadas a funciones.

Ahora se han sofisticado pareciéndose bastante a lo que podrían ser llamadas a APIs, pero con un mayor espectro.

Permiten al LLM interactuar con diferentes servicios, contenidos, etc.

All tools

Name	Description
ADS4GPTs	Integrate AI native advertising into your Agentic application.
AgentQL	AgentQL tools provides web interaction and structured data extraction...
AINetwork Toolkit	AI Network is a layer 1 blockchain designed to accommodate large-scal...
Alpha Vantage	Alpha Vantage Alpha Vantage provides realtime and historical financia...
Amadeus Toolkit	This notebook walks you through connecting LangChain to the Amadeus L...
Apify Actor	Apify Actors are cloud programs designed for a wide range of web scra...
ArXiv	This notebook goes over how to use the arxiv tool with an agent.
AskNews	AskNews infuses any LLM with the latest global news (or historical ne...
AWS Lambda	Amazon AWS Lambda is a serverless computing service provided by Amazo...
Azure AI Services Toolkit	This toolkit is used to interact with the Azure AI Services API to ac...
Azure Cognitive Services Toolkit	This toolkit is used to interact with the Azure Cognitive Services AP...
Azure Container Apps dynamic sessions	Azure Container Apps dynamic sessions provides a secure and scalable ...
Shell (bash)	Giving agents access to the shell is powerful (though risky outside a...
Bearly Code Interpreter	Bearly Code Interpreter allows for remote execution of code. This mak...
Bing Search	Bing Search is an Azure service and enables safe, ad-free, location-a...
Brave Search	This notebook goes over how to use the Brave Search tool.
BrightDataWebScraperAPI	Bright Data provides a powerful Web Scraper API that allows you to ex...
BrightDataSERP	Bright Data provides a powerful SERP API that allows you to query sea...
BrightDataUnlocker	Bright Data provides a powerful Web Unlocker API that allows you to a...
Cassandra Database Toolkit	Apache Cassandra® is a widely used database for storing transactional...
CDP	The CDP Aagentkit toolkit contains tools that enable an LLM agent to i...

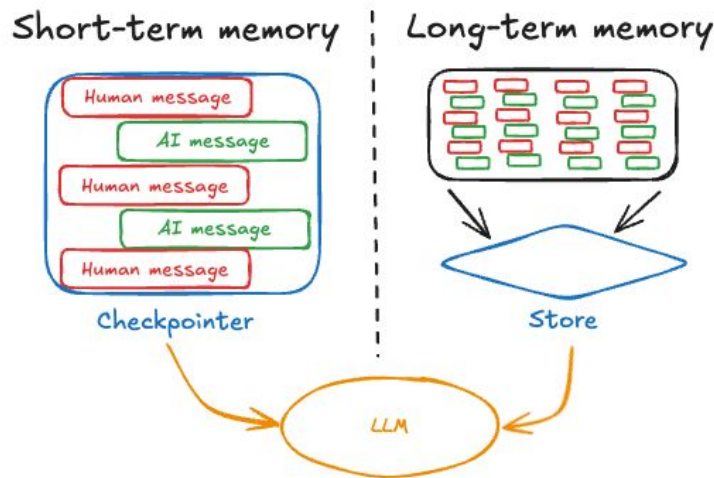




Memory

Es el mecanismo que les permite **almacenar, recuperar y actualizar información** más allá de la ventana de contexto inmediata del modelo

Permite dar continuidad a las ejecuciones, tener en cuenta resultados anteriores por ejemplo para personalización, y eventualmente eficiente la utilización de tokens,



El LLM puede decidir entre dos o más rutas posibles.

El LLM puede decidir cuál de varias herramientas utilizar.

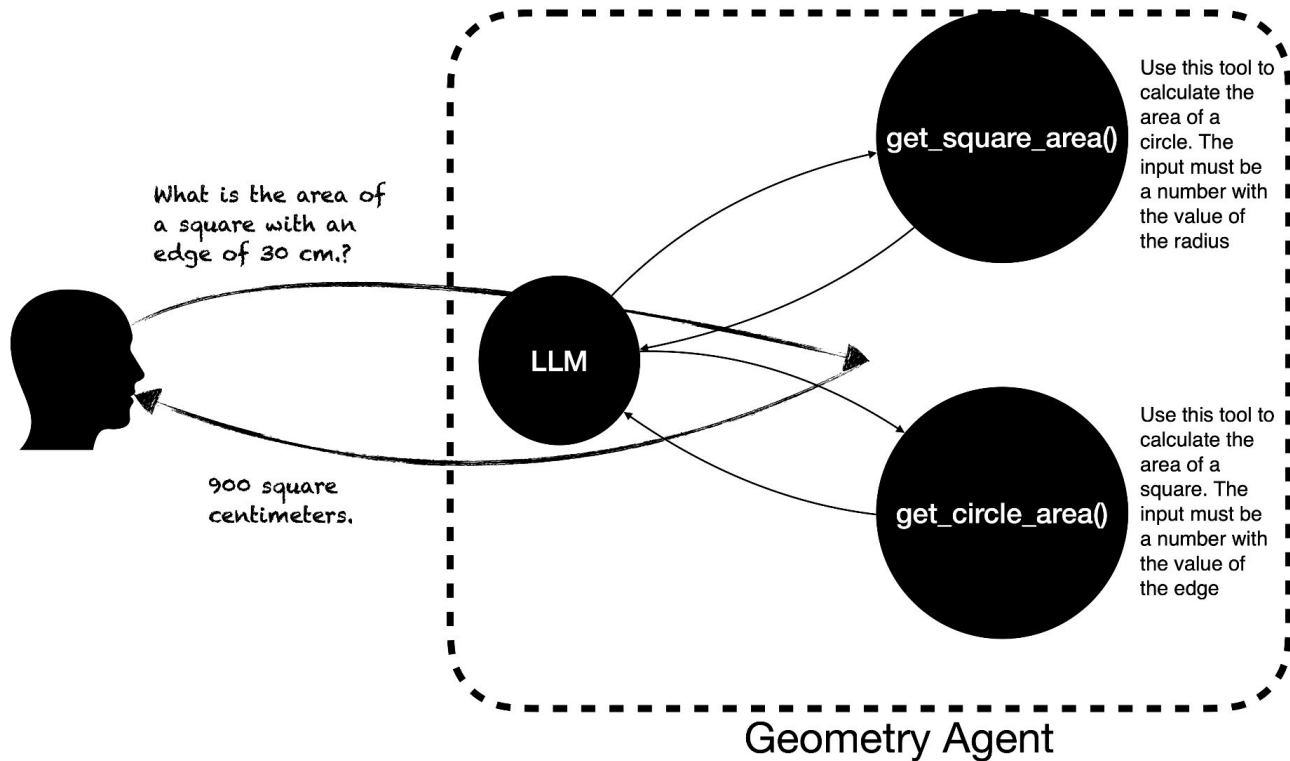
El LLM puede determinar si la respuesta generada es suficiente o si necesita más iteraciones.

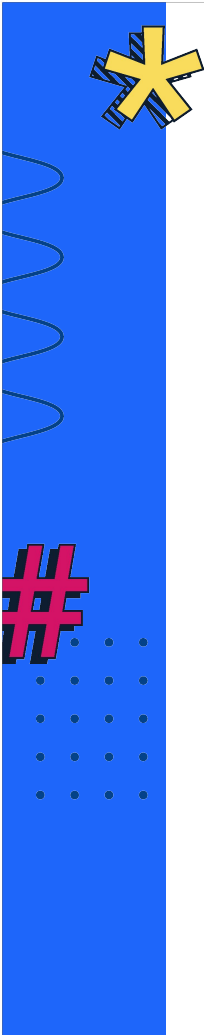
Multi-agente: varios agentes colaboran entre sí.

Agente autónomo

**LLM level
of control**

SIMPLE TOY DEMO

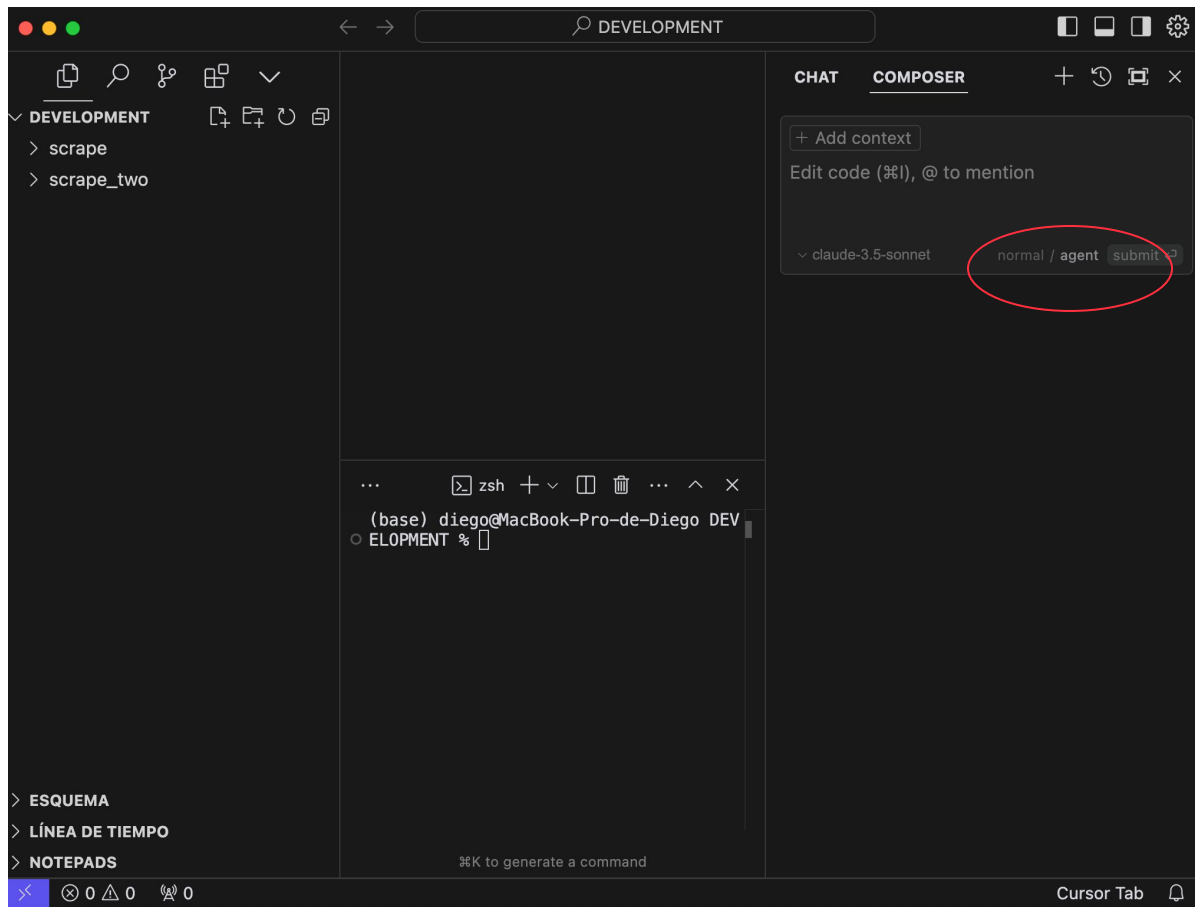




Ejemplos prácticos







Demos



Cursor Agent

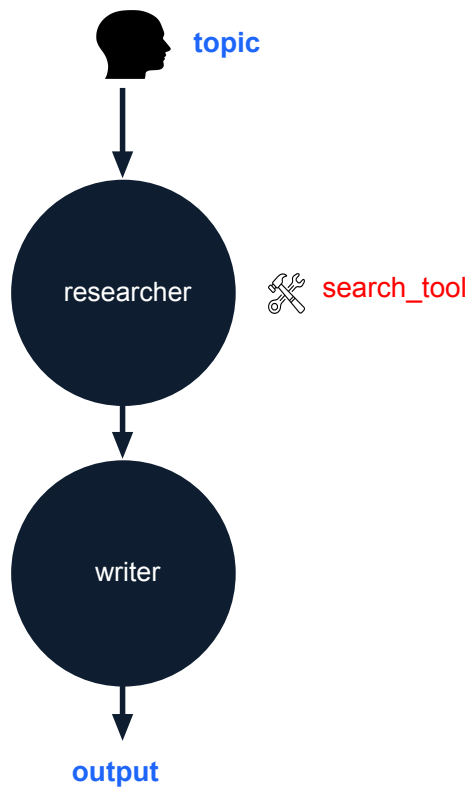


Seo Research

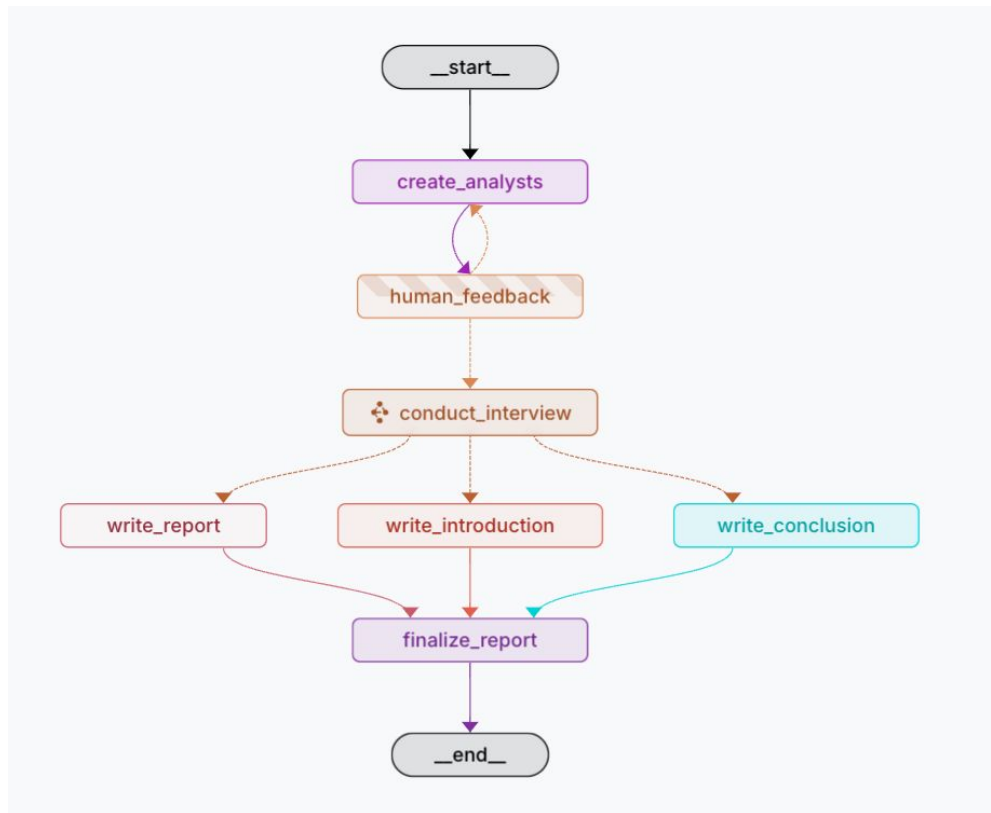


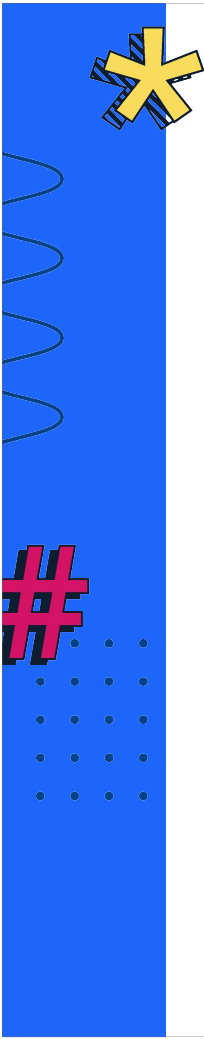
GPT Research

Seo Research



Research assistant





MCPs

MODEL CONTEXT PROTOCOL FUNDAMENTOS APLICACIONES



formacion@thebridgeprotocol.es
16 de mayo de 2025

1. INTRODUCCIÓN A MCP
2. ARQUITECTURA Y COMPONENTES
3. INSTRUCCIONES Y CONTEXTO
4. IMPLEMENTACIÓN PRÁCTICA
5. CASOS DE USO
6. FUTURO Y EVOLUCIÓN DEL MCP

Un estándar abierto que
mejora la comunicación
entre aplicaciones y
modelos de lenguaje.

- Define un formato estructurado para mensajes entre aplicaciones y LLMs
- Permite añadir metadatos y contexto a las solicitudes
- Facilita la creación de aplicaciones más potentes y consistentes
- Diseñado para un grupo de expertos en IA para unificar las comunicaciones con modelos



¿QUÉ ES EL MODEL CONTEXT PROTOCOL?

PROBLEMA QUE RESUELVE MCP

Las desventajas de aplicaciones con LLMs se enfrentan a varios desafíos:

- Cada proveedor de modelos utiliza formatos de entidades/objetos diferentes
- Falta de estructura estándar para los mensajes
- Dificultad para transferir metadatos y contexto adicional
- Complicaciones para generar instrucciones específicas del modelo
- Inconsistencia en la experiencia del usuario entre diferentes modelos
- Dificultad para jugar entre proveedores de modelos

El MCP estandariza estas comunicaciones, aportando estructura, consistencia y portabilidad.



BENEFICIOS DEL MODEL CONTEXT PROTOCOL

- **Interoperabilidad:** Facilita la comunicación entre aplicaciones y múltiples modelos de lenguaje
- **Portabilidad:** Reduce la dependencia de un único proveedor de modelos
- **Extensibilidad:** Define el estándar base de mensajes y su estructura
- **Evolución:** Permite añadir nuevas funcionalidades manteniendo compatibilidad



ARQUITECTURA DEL MCP



COMPONENTES PRINCIPALES DEL MCP

Los bloques fundamentales del MCP son:

- **Instructions:** Directrices globales para el modelo
- **Conversation:** Secuencia de mensajes interactivos
- **Message:** Unidad individual de comunicación
- **Content:** El contenido real de cada mensaje
- **Content Element:** Información adicional estructurada



MENSAJES EN MCP

Un mensaje en MCP contiene varios elementos clave:

```
{
  "role": "user",
  "content": "¿Qué es el protocolo MCP de Bridge?"
}
```

Cada mensaje representa una unidad de comunicación en la conversación.



ROLES EN MCP

El MCP define tres roles principales:

- **User:** Representa al usuario final que interactúa con el modelo
- **Assistant:** Representa al modelo de lenguaje respondiendo
- **System:** Proporciona instrucciones generales o contexto al modelo



Cada rol tiene un propósito específico en la conversación y ayuda al modelo a entender el contexto de los mensajes.



INSTRUCCIONES EN MCP

Las instrucciones son directrices globales para el modelo y pueden contener:

- Descripción de la tarea a realizar
- Tono y estilo de respuesta deseado
- Restricciones o limitaciones
- Comentarios específicos relevantes
- Comportamiento esperado ante ciertos tipos de consultas

```
{
  "instructions": "Responde de manera concisa y directa."
}
```

Las instrucciones son independientes de los mensajes individuales y abarcan toda la conversación.



ELEMENTOS DE CONTEXTO

Los elementos de contexto permiten enriquecer los mensajes con información adicional:

- **Documentos:** Contenido externo referenciado
- **Imágenes:** Representaciones visuales
- **Datos estructurados:** Tablas, gráficos, JSON, etc.
- **Archivos:** Referencias a recursos externos
- **Código:** Fragmentos de código en diversos lenguajes
- **Metadatos:** Información sobre el mensaje o su contenido

Estos elementos proporcionan información adicional al modelo para generar respuestas más precisas y relevantes, mejorando significativamente la capacidad para entender el contexto completo de la consulta.



TIPOS DE CONTENIDO EN MCP

El MCP soporta múltiples tipos de contenido en los mensajes:



- **Text:** Contenido textual básico
- **Image:** Representaciones visuales
- **Document:** Textos más extensos y estructurados
- **Code:** Información estructurada en diversos formatos
- **Audio:** Clips de audio o grabaciones de voz
- **Video:** Contenido audiovisual

Esta flexibilidad permite comunicaciones más ricas y variadas.



IMPLEMENTACIÓN DE MCP

Pasos para implementar MCP en una aplicación:

1. Estructurar la conversación siguiendo el formato MCP
2. Definir instrucciones globales para el modelo
3. Crear mensajes con los datos apropiados
4. Añadir elementos de contexto relevantes
5. Enviar la conversación al modelo de lenguaje
6. Procesar la respuesta del modelo
7. Actualizar la conversación con la nueva información



SERVIDORES MCP

Un servidor MCP actúa como intermediario entre las aplicaciones cliente y los modelos de lenguaje:

- **Recibe solicitudes** en formato MCP de las aplicaciones cliente
- **Traduce** estas solicitudes al formato específico del modelo seleccionado
- **Envía** la solicitud procesada al modelo de lenguaje
- **Recibe** la respuesta del modelo
- **Convierte** la respuesta al formato MCP
- **Devuelve** la respuesta a la aplicación cliente

Los servidores MCP pueden gestionar múltiples modelos y proveedores, permitiendo intercambios de forma transparente para el cliente. También pueden implementar funcionalidades adicionales como caché, balanceo de carga, monitorización y estadísticas de uso.



CONFIGURACIÓN DE SERVIDOR MCP

Aspectos clave para configurar un servidor MCP:

- Selección de modelos soportados
- Configuración de API keys para cada proveedor
- Definición de endpoints para cada modelo
- Políticas de rate limiting para controlar el uso
- Mecanismos de caché para optimizar respuestas
- Exponer de autorización para acceso seguro
- Opciones de logging para depuración y monitoreo

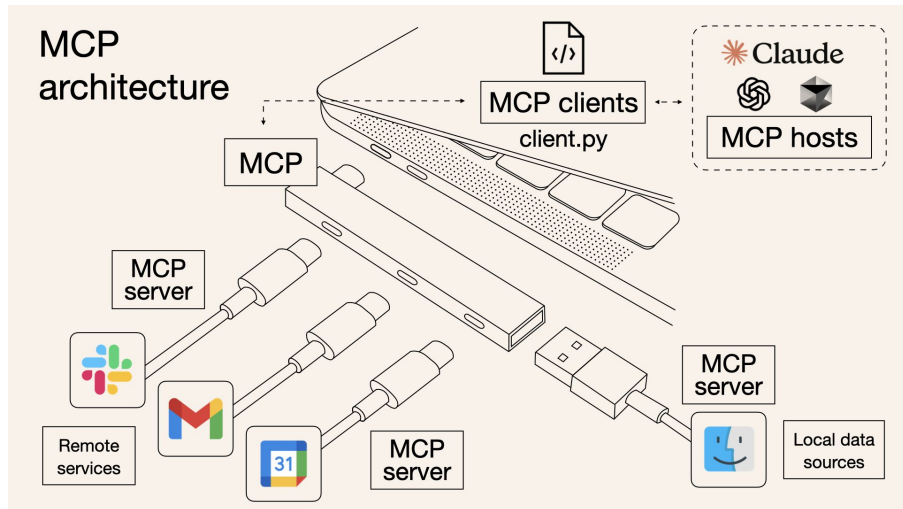
```
{
  "models": {
    "openai": {
      "api_key": "sk-...",
      "url": "https://api.openai.com/v1"
    },
    "anthropic": {
      "api_key": "sk-...",
      "url": "https://api.anthropic.com/v1"
    }
  }
}
```



MCP

“MCP es un protocolo abierto que estandariza cómo las aplicaciones proporcionan contexto a los modelos de lenguaje (LLM).”

Piensa en MCP como un puerto USB-C para las aplicaciones de inteligencia artificial. Así como el USB-C ofrece una forma estandarizada de conectar tus dispositivos a diversos periféricos y accesorios, MCP proporciona una forma estandarizada de conectar los modelos de IA a diferentes fuentes de datos y herramientas.”



REST API in Web Development

Conecta web apps a servicios y datos

=

MCP in AI Ecosystem

Conecta AI apps a herramientas y datos

Open Standard

Un protocolo diseñado para permitir la adopción y extender el uso

Tool Integration

Conexión segura de aplicaciones IA con herramientas y capacidades

Acceso a datos

Acceso controlado a recursos de la aplicación y datos contextuales

Arquitectura Cliente Servidor

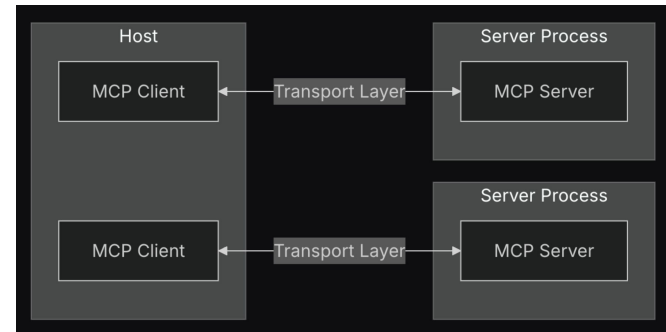
La comunicación se realiza a través de capa de transporte:

- **Stdio** para procesos locales
- **HTTP** con **SSE**
 - POST para mensajes de cliente a servidor

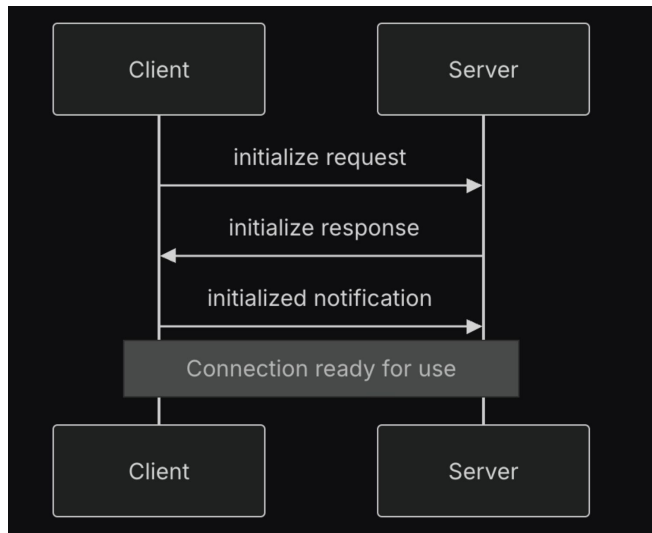
MCP implementa 4 tipos de mensajes:

- **Requests:** Esperan una respuesta
- **Results:** Respuestas satisfactorias
- **Errors:** Respuestas fallidas
- **Notifications:** Mensajes en un sólo sentido que no requieren respuesta

Todas las comunicaciones basadas en JSON RPC 2.0



Lifecycle

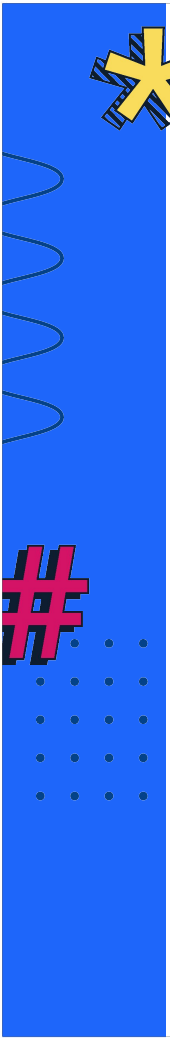


Esta asociación es independiente de la autenticación y autorización
Soporte para OAuth

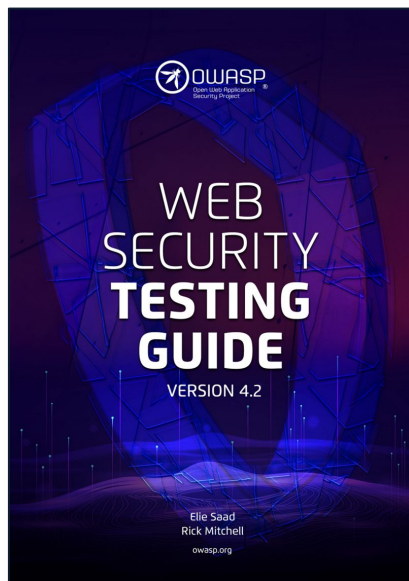
Resources

Tools

Prompts



Ejemplos prácticos



Best Practices

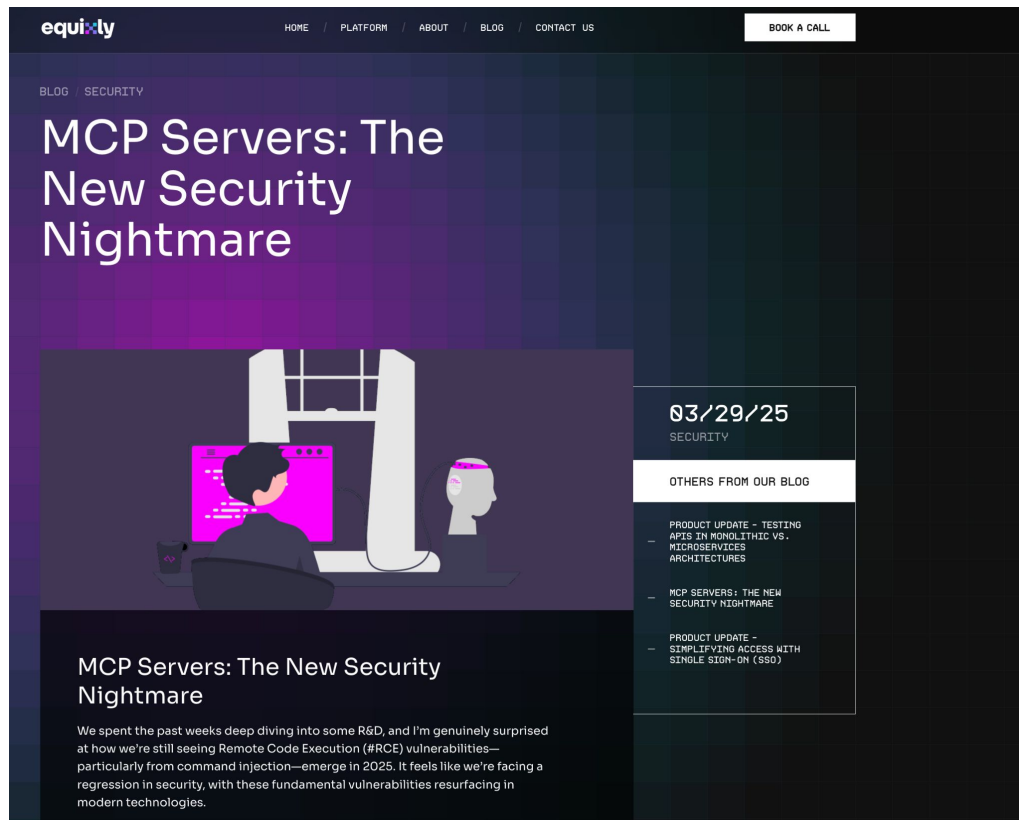
Here are some best practices to enhance the security of your MCP server:

1. **Always validate Origin headers** (enabled by default)
2. **Use authentication** for all MCP endpoints in production
3. **Deploy behind HTTPS** in production environments
4. **Keep your auth_token secret** and rotate it regularly
5. **Implement proper error handling** to avoid leaking sensitive information
6. **Validate inputs thoroughly** in your tool implementations
7. **Implement rate limiting** for MCP endpoints to prevent abuse
8. **Follow the principle of least privilege** when implementing tools
9. **Log security events** to detect and respond to potential security incidents

<https://github.com/punkpeye/awesome-mcp-servers>

<https://github.com/TensorBlock/awesome-mcp-servers/blob/main/docs/security.md>





Common Vulnerability Pattern: Command Injection

Let's examine a common pattern discovered in our research. Consider this actual implementation code from a popular MCP server:

```
dispatch_user_alert.py

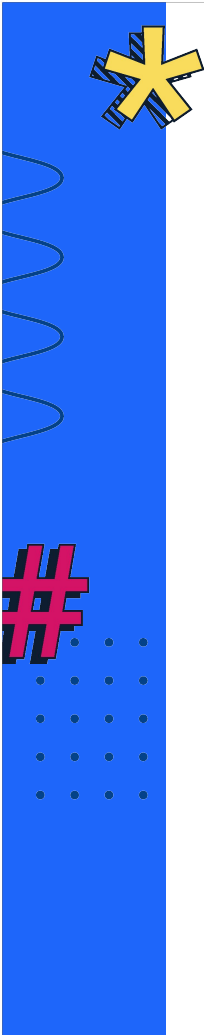
def dispatch_user_alert(self, notification_info: Dict[str, Any], summary_msg: str) -> bool:
    """Sends system alert to user desktop"""
    notify_config = self.user_prefs["alert_settings"]

    try:
        alert_title = f"{notification_info['title']} - {notification_info['severity']}"
        if sys.platform == "linux":
            subprocess.call(["notify-send", alert_title])
        return True
```

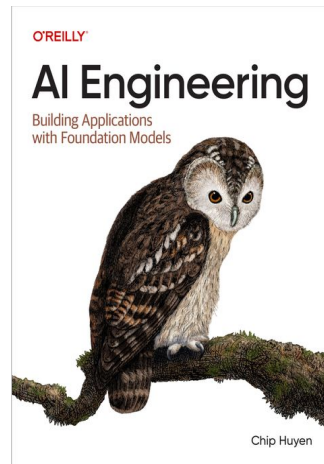
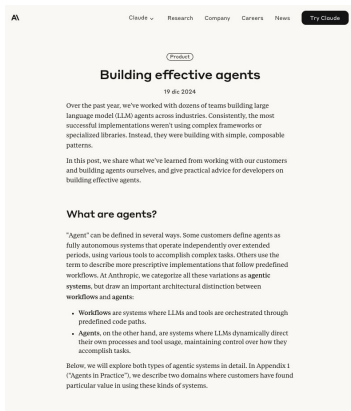
This code contains a classic command injection vulnerability. An attacker could craft a payload in the notification_info dictionary with values containing shell metacharacters.

<https://equixly.com/blog/2025/03/29/mcp-server-new-security-nightmare/>



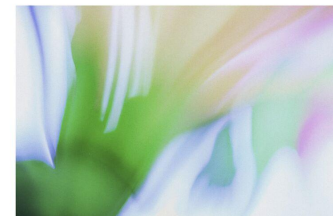


¿Por dónde empezar?



OpenAI

A practical guide to building agents



DeepLearning.AI

Explore Courses AI Newsletter AI Dev 25 Community Company Start Learning

AI Courses > Short Courses > Multi AI Agent Systems with crewAI

Short Course Beginner 2 Hours 42 Minutes

Multi AI Agent Systems with crewAI

Instructor: Jobo Moura

Enroll for Free



Multi AI Agent Systems with crewAI

Jobo Moura

crewAI

What you'll learn

- Exceed the performance of prompting a single LLM by designing and prompting a team of AI agents through natural language.
- Use an open source library, crewAI, to automate repeatable, multi-step tasks like tailoring a resume to a job description, and automate business processes that are typically done by a group of people, like event planning.
- By creating a team of AI agents, you can define a specific role, goal, and backstory for each agent, which breaks down complex multi-step tasks and assigns them to agents that are customized to perform those tasks.

Multi AI Agent Systems with crewAI

Beginner

2 Hours 42 Minutes

19 Video Lessons

7 Code Examples

Instructor: Jobo Moura

crewAI

Enroll for Free

LangChain Academy

Level up with LangChain Academy

Dive into self-paced, comprehensive courses designed to help you build relevant skills and knowledge to succeed with LangChain products.



LangChain Academy New Course: Introduction to LangSmith

Enroll Now

For us: 10 min

Our Courses

LangSmith

Introduction to LangSmith

Course

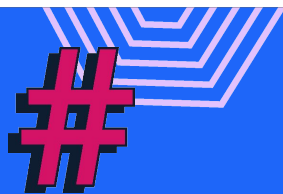
Learn the essentials of LangSmith - our

LangGraph

Introduction to LangGraph

Courses

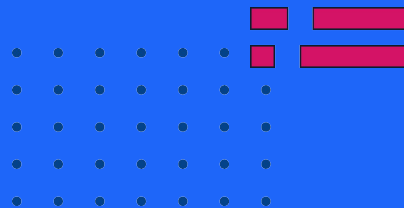
Learn the basics of LangGraph - our

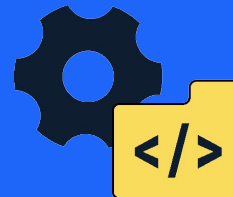
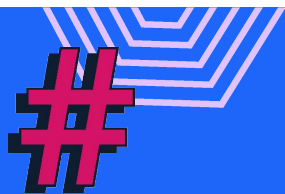


{codemotion}

CONFERENCE
MADRID 2025

**Don't forget to
rate the talk!**





{codemotion}

CONFERENCE
MADRID 2025

Thanks!

